Ders 6 Genişletilmiş Notu: Command, Iterator, Memento ve Observer Kalıpları

Ders 6 Genişletilmiş Notu: Command, Iterator, Memento ve Observer Kalıpları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Command Kalıbı: Amaç ve Motivasyon
- 2. Command Kalıbının Yapısı ve Receiver'ın Verilmesi
- 3. Command Uygulama Örneği: Akıllı Ev Kumandası
- 4. Iterator Kalıbı: Amaç ve Motivasyon
- 5. Iterator İçin İki Çözüm
- 6. Iterator Kalıbının Yapısı ve Gerçekleme Kararları
- 7. Iterator Uygulama Örneği: İki Yemek Firmasının Menüleri
- 8. Memento Kalıbı: Amaç ve Motivasyon
- 9. Memento Kalıbının Yapısı ve Gizlilik Sorunu
- 10. Memento: İç Sınıf ile Çözüm ve Derste Denenen Açık
- 11. Memento Uygulama Örneği: Doküman Undo/Redo
- 12. Observer Kalıbı: Amaç ve Yapı
- 13. Observer Gerçekleme Ayrıntıları
- 14. Observer Uygulama Örneği: Müşteri İlişkileri Programı
- 15. UML Şemalarının Çizilmesi
- Örnek Sorular
- Dersin Toparlaması

1. Command Kalıbı: Amaç ve Motivasyon

Ders, önceki derste başlanan davranışsal (behavioral) kalıplarla devam eder ve ilk olarak Command kalıbı işlenir.

1.1 Amaç

Bir nesneye normalde nasıl istek göndeririz? Nesnenin bir metodunu çağırırız; metot çalışır, isteğin gereğini yapar, belki bir sonuç döndürür ya da aktarılan parametreyi değiştirir. Command kalıbında ise metot çağrıları (istekler) birer nesneye dönüştürülür: giden istekler nesne hâline getirilir.

Bunu niçin isteriz? Hocanın aklına gelen ilk amaç mesajları kuyruğa koymaktır. Asenkron çalışıldığını, bir metot çağrısının yanıtının geç hesaplanabileceğini düşünün. Çağrıları sıra sıra işlemek yerine bir kuyruk mantığına koymak isteyebiliriz. Ya da her metot çağrısının bir günlüğünü (log) tutmak isteyebiliriz. Bunlar son derece makul isteklerdir.

Kalıp olmadan bu işler iki yolla yapılabilir:

- İlgili metot çağrılarının hepsinden önce ya da sonra bu işleri yapacak ek kodu uygulama mantığının içine yazmak.

- Aspect Oriented Programming (cepheye yönelik programlama) gibi, kodu çalışma anında değiştirerek çeşitli "aspect"leri (ör. günlük tutma) otomatik yedirmeyi sağlayan bir yaklaşım kullanmak. Bu yaklaşım kendine özel bir dil ve kendine özel bir sanal makine gerektirir.

Normal nesneye dayalı yaklaşımla çalışırken böyle ihtiyaçlar doğduğunda kullanılabilecek kalıp Command'dır.

1.2 GoF örneği: menü komutları (ekrandan, 00:10:00)

Bir GUI kütüphanesi hazırlarken üstte bir dosya menüsü, onun çeşitli seçenekleri, onların alt seçenekleri ve en sonunda bir komut yürüten yaprak düğümler bulunur. Aynı işler bir düğmenin tıklama (on click) olayına da bağlanabilmelidir. Araya bir soyutlama katmanı konursa bu iş rahatça yapılır: menü öğesinin tıklama olayında yapılacak tek şey ilgili komutun yürütülmesidir. Komut bir arayüz (ya da soyut sınıf) olarak tanımlanır ve bu arayüze bir metot konur.

Slayttaki şema:

```
Command {Execute()}                        (soyut)
OpenCommand {Execute(), AskUser()} ⟶▷ Command
OpenCommand ⟶application⟶► Application {Add(Document)}

OpenCommand.Execute():
   name = AskUser()
   doc = new Document(name)
   application->Add(doc)
   doc->Open()
```

Bu arayüzü gerçekleyen somut sınıflar hazırlanır; bu sınıflardan oluşturulan nesneler "nesneleştirilmiş komutlar"dır. Örneğin açma komutu (`OpenCommand`), `Execute` metodunun içini doldurarak dosya açma işini yürütür.

Hoca burada önceki şemadan bir noktaya dikkat çekti: parça-bütün ilişkisinde parçanın bütünü de bilmesi gerekebilir. Açma komutu çalıştıktan sonra açtığı dokümanı uygulamanın dokümanları arasına ekleyebilmelidir; bunun için komuttan uygulamaya doğru bir bağlantı gerekir. Her senaryoda bu geri yöndeki ilişkiye ihtiyaç olmayabilir, ama olabileceği için kalıp kurulurken bu da düşünülmelidir.

Başka bir örnek: uygulama birden fazla dokümanla çalışıyor olabilir ve birinde kopyala-yapıştır yapılacaktır. Komutlar nesneleştirildiği için, kopyalama komutunun fareyle mi klavye kısayoluyla mı tetiklendiği fark etmez; komut istenen yerle ilişkilendirilebilir. Bir `PasteCommand`'ın `Execute` metodu dokümanın `Paste` metodunu kullanarak yapıştırma yapar; burada da komuttan dokümana doğru ilişki gerekir.

2. Command Kalıbının Yapısı ve Receiver'ın Verilmesi

2.1 Kalıp yapısı (ekrandan, 00:11:20)

```
Command {Execute()}                        (soyut)
ConcreteCommand {Execute(), state} ⟶▷ Command        (kalıtım)
ConcreteCommand ⟶receiver⟶► Receiver {Action()}
   Execute():  receiver->Action();
Invoker ◇⟶► Command                        (toplama)
Client ⟶► Receiver
Client ⋯⋯► ConcreteCommand                 (bağımlılık)
```

Her komut ortak bir arayüz altında toplanır ve bu arayüz en az bir ortak metot imzasına (`Execute`) ihtiyaç duyar. Somut komutlar kodlanır; `Execute` metodunun içinde, isteğin alıcısının (Receiver) yapması gereken eylem (`Action`) yapılır.

2.2 Receiver komuta nasıl verilir?

Receiver her zaman Invoker ile aynı varlık olmayabilir. Dosya açma örneğinde Invoker (uygulama) aynı zamanda Receiver'dı, ama bunlar farklı varlıklar da olabilir. Receiver farklı olabilecekse komuta verilebilmesi gerekir. Hocanın tartıştığı seçenekler:

- **`Execute` metoduna parametre olarak vermek.** Ancak bu parametre, Receiver'ın Invoker'dan farklı olmadığı durumlarda kullanılmayacaksa bu bir tasarım zayıflığı sayılır: aktarılan ama hiç kullanılmayan bir parametre olur.
- **Arayüze bir `setReceiver` metodu eklemek** ve somut komutlarda gerçeklemek. Üstteki arayüz soyut sınıfa çevrilebiliyorsa Receiver'ın yönetim kodu doğrudan o sınıfa yazılabilir.
- **Somut komutlara farklı yapıcılar yazmak.** Receiver bazı komutlarda gerekip bazılarında gerekmiyorsa, üst sınıfa/arayüze bir şey eklenmez; kimi somut komut Receiver parametresi alan, kimi almayan yapıcıyla oluşturulur. Bu iş yalnız komutun ilk oluşturulduğu anda yapıldığı için büyük bir sakınca değildir.

Özetle: Receiver'ın aktarılmasını üst sınıfta/arayüzde tanımlamak bir sorun doğurmayacaksa orada tanımlayın, doğuracaksa alttaki somut komutlarda yapın.

3. Command Uygulama Örneği: Akıllı Ev Kumandası

3.1 Problem (ekrandan, 00:14:40)

Hoca bu örneği çok beğenmediğini ama "örneğin olmamasından iyidir" diyerek verdi; Head First Design Patterns kitabındaki örnekten esinlenmiştir.

Slayttaki problem: Bir akıllı ev yazılımı üretiyoruz. Yazılımda basit bir açma/kapama kumandası ekranı var. Kumanda ekranındaki aç/kapa düğmelerine odaların ışıklarını, televizyonları vb. açma/kapama işlevleri kazandıracağız. Bir düğmeye oda ışığı, birine televizyon, birine ısıtıcı, birine soğutucu atanacak ve farklı senaryolarda farklı işler yürütülecek.

Kumandada yapılacak işler: belli numaralı bir düğmeyi açmak, kumandaya yeni bir düğme eklemek, belli bir düğmeyi kapatmak ve bütün düğmeleri kapatmak (ör. evden çıkarken).

3.2 Tasarım

Slayttaki sınıf şeması metne dökülürse:

```
«interface» AcKapa
   + ac(): boolean
   + kapa(): boolean

AydinlatmaDugmesi ┄┄▷ AcKapa      - isim: String
   + ac(): boolean, + kapa(): boolean, + toString(): String, + AydinlatmaDugmesi(isim: String)
ElektronikAlet ┄┄▷ AcKapa         - isim: String
   + ac(): boolean, + kapa(): boolean, + ElektronikAlet(isim: String), + toString(): String
Iklimlendirme ┄┄▷ AcKapa          - isim: String
   + ac(): boolean, + kapa(): boolean, + Iklimlendirme(isim: String), + toString(): String

Kumanda ──*──► AcKapa
   - acKapaDugmeleri: ArrayList<AcKapa>
   + ac(dugmeNo: int): boolean
   + Kumanda()
   + dugmeEkle(dugme: AcKapa)
   + kapa(dugmeNo: int): boolean
   + hepsiniKapa()

AnaProgram ──► Kumanda
   - kumanda: Kumanda
   + AnaProgram()
   + evBoskenSokakKapisiAcildi()
   + evdeKimseKalmadi()
   + main(args: String[])   (statik)
```

Tasarımla ilgili açıklamalar:

- GoF yapısında yalnız bir `Execute` vardı; burada yapılabilecek iki farklı iş olduğu için arayüzde iki metot (`ac`, `kapa`) vardır. Tek bir `execute` metoduna "açayım mı, kapayayım mı" parametresi vermek de olurdu. Metot imzasını ihtiyaca göre tanımlamakta hiçbir sakınca yoktur.
- Arayüzü gerçekleyen ayrı sınıflar (aydınlatma düğmesi, elektronik alet, iklimlendirme cihazı) vardır, çünkü gerçekte bunları açıp kapatmak farklı kodlar gerektirir: bir ev otomasyon yazılımında SCADA ile mi kod yazılacağı, akıllı cihazların açık API'leri üzerinden mi metot çağrılacağı cihazdan cihaza değişir. Yapılan iş "açma" ya da "kapama" olsa da aydınlatmayı açmak, elektronik aleti açmak ve iklimlendirme cihazını açmak farklı komutlardır.
- `Kumanda`'da `ArrayList` seçilmesinin nedeni, belli numaralı bir düğmeye doğrudan erişebilmektir; bu seçim kalıpla ilgili bir bilgi taşımaz.
- Düğme zaten listedeyse tekrar eklememe gibi denetimler bilerek yapılmamıştır; bunlar kalıbı öğretmeye yönelik özellikler değildir.

3.3 Kod (ekrandan, 00:17:40, 00:19:20, 00:20:00)

```
package dp.command.example;
public interface AcKapa {
    public boolean ac( );
    public boolean kapa( );
}

package dp.command.example;
public class AydinlatmaDugmesi implements AcKapa {
    private String isim;
    public AydinlatmaDugmesi(String isim) { this.isim = isim; }
    public String toString( ) { return "AcKapa:" + isim; }
    public boolean ac() { return true; }
    public boolean kapa() { return true; }
}

package dp.command.example;
import java.util.*;
public class Kumanda {
    private ArrayList<AcKapa> acKapaDugmeleri;
    public Kumanda( ) { acKapaDugmeleri = new ArrayList<AcKapa>(); }
    public void dugmeEkle( AcKapa dugme ) { acKapaDugmeleri.add(dugme); }
    public boolean ac( int dugmeNo ) {
        if( acKapaDugmeleri.get(dugmeNo) == null )
            return false;
        return acKapaDugmeleri.get(dugmeNo).ac();
    }
    public boolean kapa( int dugmeNo ) {
        if( acKapaDugmeleri.get(dugmeNo) == null )
            return false;
        return acKapaDugmeleri.get(dugmeNo).kapa();
    }
    public void hepsiniKapa( ) {
        for( AcKapa dugme : acKapaDugmeleri )
            dugme.kapa();
    }
}

public class AnaProgram {
    Kumanda kumanda;
    public AnaProgram( ) {
        kumanda = new Kumanda();
```

```
        kumanda.dugmeEkle( new AydinlatmaDugmesi( "Antre" ) );
        kumanda.dugmeEkle( new AydinlatmaDugmesi( "Salon" ) );
        kumanda.dugmeEkle( new AydinlatmaDugmesi( "Mutfak" ) );
        kumanda.dugmeEkle( new AydinlatmaDugmesi( "Oda" ) );
        kumanda.dugmeEkle( new ElektronikAlet( "Vestel TV" ) );
        kumanda.dugmeEkle( new Iklimlendirme( "Kombi" ) );
    }
    public void evBoskenSokakKapisiAcildi( ) {
        kumanda.ac(0); kumanda.ac(5);
    }
    public void evdeKimseKalmadi( ) { kumanda.hepsiniKapa( ); }
    public static void main(String[] args) {
        AnaProgram prg = new AnaProgram();
        prg.evBoskenSokakKapisiAcildi();
        System.out.println("Evden çıkılıyor");
        prg.evdeKimseKalmadi();
    }
}
```

`ElektronikAlet` ve `Iklimlendirme` sınıflarının kodu `AydinlatmaDugmesi` ile aynı kalıptadır (slaytta ayrıca gösterilmedi). Açma ve kapama basitçe `true` döndürür; yani başarısızlık durumu bu örnekte düşünülmemiştir. `toString` yalnız düğmenin adını içeren bir metin döndürür. Hoca, "açıksa şöyle yap, kapalıysa böyle yap" gibi eklemelerin kalıbı öğretmeye katkı sağlamayacağı için örneği bilerek basit tuttuğunu söyledi. Düğmenin açık mı kapalı mı olduğunu öğrenmek için bir `getState` metodu koymayı unuttuğunu da belirtti.

`Kumanda.ac` metodu, belli numarada düğme yoksa `false` döndürür; varsa ilgili düğmenin `ac` metodunun döndürdüğünü döndürür. `kapa` da benzerdir. `hepsiniKapa` bütün düğmelerin `kapa` metodunu çağırır.

> **Ders sonrası not:** Kodda düğmenin olmadığı durum `acKapaDugmeleri.get(dugmeNo) == null` ile denetleniyor. Java'da `ArrayList.get`, listenin boyutunun dışındaki bir indeks için `null` döndürmez, `IndexOutOfBoundsException` fırlatır (küçük bir Java denemesiyle doğrulandı). Hocanın amaçladığı davranış için denetim `dugmeNo < 0 || dugmeNo >= acKapaDugmeleri.size()` biçiminde yazılmalıdır.

Ana programda kumanda tanımlanır ve düğmeler eklenir: dört aydınlatma düğmesi (antre, salon, mutfak, oda), bir elektronik alet (televizyon) ve bir iklimlendirme cihazı (kombi). "Ev boşken sokak kapısı açıldı" senaryosunda 0 numaralı düğme (girişteki ışık) ve 5 numaralı düğme (kombi) açılır; hoca bu örneği yazarken mevsimin sonbahar ya da kış olduğunu, bu yüzden girişte kombinin açılmasını tercih ettiğini söyledi. "Evde kimse kalmadı" senaryosunda bütün düğmeler kapanır. Hoca programı çalıştırdığında girişteki ışığın ve kombinin açıldığını, evden çıkarken eklenen bütün düğmelerin kapandığını gösterdi.

3.4 Örneğin geliştirilmesi: termometre (derste sözlü, 00:20:44)

Hoca, örneği çeşitlendirmek için bir termometre cihazı eklenebileceğini düşündü. Termometreden ölçüm almak, şimdiye kadarki "açabildiyse `true`, açamadıysa `false`" döndüren komutlardan farklıdır. Kalıp yapısında bu, Receiver'ın bir eyleminin (action) olabilmesine karşılık gelir. Hocanın notu: termometre ölçüm sonucunun (derece) geri döndürülmesi Receiver nesnesinin kullanılmasıyla yapılabilir; böylece örnek biraz daha geliştirilebilir.

4. Iterator Kalıbı: Amaç ve Motivasyon

4.1 Amaç

Iterator, güncel programlama dillerinde zaten yerini bulmuş bir kalıptır; birçok dilde kendiniz kurmak zorunda değilsiniz. Yine de nasıl çalıştığını bilmek farklı bir bakış açısı kazandırır.

Kalıp, parça-bütün yapısındaki ya da farklı parçalardan oluşan bir bileşenin parçalarını istenen bir biçimde dolaşmayı sağlar. Bunu yaparken bileşenin iç yapısını açığa çıkarmaz. Başka bir deyişle "şu anda şu düğümdeyim, bu düğümün türü şu, sınıfı şu, özellikleri şunlar" gibi

ayrıntılar kullanıcıdan soyutlanır. Böylece hem parçaların iç yapısını dışarı açmak zorunda kalmazsınız hem de dolaşırken yalnız izin verilen işlerin yapılmasını sağlayarak daha güvenli çalışırsınız.

4.2 Örnek: generic yapı yokken liste dolaşmak

Bir veri yapısının içinde dolaşmak istiyoruz, fakat henüz generic yapımız yok diyelim. Bir liste düşünün: düğümleri ne türden veri içerecek? Sayı mı, karakter mi, herhangi bir sınıftan herhangi bir nesne mi? Java'daki "kozmik" `Object` üst sınıfı olmasaydı ne olurdu? C++'ın eski standartlarında bütün sınıfların ortak üst sınıfı yoktur; şablon sınıf (template class) özelliği de eski standartlarda yoktu. Bu durumda tam sayı içeren liste için ayrı, öğrenci nesneleri içeren liste için ayrı dolaşma metodu yazmak gerekir; yalnız tür adı değişir, gerisi aynıdır. Kodun taşınabilirliğini artırmak için farklı standartlarda desteklenmeyen özelliklerden kaçınılıyorsa ve daha genel bir çözüm isteniyorsa Iterator kalıbını gerçeklemek işe yarar.

4.3 Java'nın iterator'ı ve KISS ilkesi

Java'daki iterator yapısı bu tasarım kalıbındaki yapıdan daha basittir. Hoca basitliğin kötü olmadığını vurguladı: işinize yarıyorsa kullanın; Java'nın iterator'ının yapabildiğinden fazlasına ihtiyacınız varsa kalıbı kurarsınız. Hoca, Java'nın iterator'ının yetmediği pratik bir ihtiyaç kendisinin de pek saptayamadığını söyledi.

Bu vesileyle KISS ilkesini hatırlattı: Keep It Simple, Stupid. Kabaca "bir işi basit tut"; bir işi olduğundan daha karmaşık tutarsak yanlış bir iş yapmış oluruz.

5. Iterator İçin İki Çözüm

5.1 Birinci çözüm (ekrandan, 00:30:00)

```
List {Count(), Append(Element), Remove(Element), ...}
ListIterator {First(), Next(), IsDone(), CurrentItem(); index} ──list──► List
```

Bir liste yapımız var: kaç eleman olduğunu döndürebilir, eleman ekleyebilir, eleman çıkarabiliriz. Bu liste üzerinde dolaşacak bir iterator sınıfı da var: listedeki ilk elemanı döndürür (`First`), sonraki elemana geçer (`Next`), hepsinin üzerinden geçilip geçilmediğini söyler (`IsDone`: hepsi dolaşılmışsa `true`, değilse `false`) ve o anki elemanı verir (`CurrentItem`). Kullanım: `IsDone` değilken `Next` diyerek liste bitene kadar dolaşılır.

- **Güçlü yönü:** Veri yapısı ile dolaşım biçimi birbirinden soyutlanmıştır.
- **Zayıf yönü:** Belli bir veri yapısı ile belli bir dolaşım biçimi arasında bağlılık oluşmuştur.

İlk etapta çok şey kazanılmamış gibi görünür, çünkü tek bağlı listede yalnız ileri gidilebilir. Ama çift bağlı listede geri de gidilebilir; dairesel (circular) listede son eleman yeniden listenin başını döndürür. Ağaçlarda da dolaşmak gerekir: ikili ağaçta (ya da ikili olmayan ağaçta) LDR (sol, veri, sağ), RDL, LRD gibi dolaşma varyasyonları vardır. Örneğin LDR'de gidebildiğin kadar sola gidilir; gidecek yer kalmayınca veri işlenir, sonra sağ çocuğa geçilir ve orada yine önce sola gidilir. Iterator bunlardan hangisiyle dolaşacak? Bunu bilmek zorunda mı? Bir tablo yapısında (anahtar-değer çiftleri) amaç yalnız bütün kayıtları dolaşmaksa sıralı mı sırasız mı dolaşıldığını bilmek gerekmez.

Bu durumda iterator'ın "listedeysem olası dolaşma biçimleri bunlar, ağaçtaysam bunlar" gibi bir ön bilgiye sahip olması gerekebilir. Bu da `List` ile `ListIterator` arasında bir bağımlılık oluşturur: liste dolaşılacaksa mutlaka `ListIterator`, ağaç dolaşılacaksa `TreeIterator` gerekir. Yapılacak iş yalnız "başka eleman kaldı mı, sıradakini getir, bir şey yapayım" ise bu iki iterator'ın farklı sınıflardan olmasının kullanıcıya bir yararı yoktur.

Slayttaki öneri: Bu bağlılığı azaltmak için genel bir liste yapısı arayüzü ve genel bir dolaşım biçimi arayüzü oluşturulabilir.

5.2 İkinci çözüm (ekrandan, 00:32:10)

```
Client ──► AbstractList {CreateIterator(), Count(), Append(Item), Remove(Item), ...}   (soyut)
Client ──► Iterator {First(), Next(), IsDone(), CurrentItem()}                        (soyut)
```

```
List ──▷ AbstractList          SkipList ──▷ AbstractList
ListIterator ──▷ Iterator      SkipListIterator ──▷ Iterator
List ┈┈► ListIterator,         ListIterator ──► List
SkipList ┈┈► SkipListIterator, SkipListIterator ──► SkipList
```

Soyut bir liste sınıfı ve soyut bir iterator sınıfı vardır (bunlar arayüz de olabilir). Soyut listeyi gerçekleyen `List` ve `SkipList` kardeş sınıflardır; hoca `SkipList`'i her elemanı dolaşmak yerine belli kriterlere uyan elemanları dolaşan bir yapı olarak örnekledi. Çift bağlı liste ya da çeşitli ağaç sınıfları da bunlara kardeş olabilir.

Arka planda `List`, `ListIterator` ile; `SkipList`, `SkipListIterator` ile ilişkilidir. Ama somut iterator'lar ortak `Iterator` arayüzünü gerçeklediği için istemci hepsine ortak biçimde erişir. Somut bir iterator'ın ek özelliklerine (ör. `SkipListIterator`'ın "yalnız şu koşula uyan kayıtları getir" komutlarına) gerçekten ihtiyaç varsa tür denetimi (type checking) ve tür dönüşümü (type casting) yapılır. Hoca tür dönüşümünün güzel bir şey olmadığını, ama burada iç mantıkta değil yalnız iterator'ın nasıl kullanılacağıyla ilgili kısımda kullanıldığı için büyük sorun oluşturmadığını söyledi.

6. Iterator Kalıbının Yapısı ve Gerçekleme Kararları

6.1 Kalıp yapısı (ekrandan, 00:40:00)

```
Client ──► Aggregate {CreateIterator()}                          (soyut)
Client ──► Iterator {First(), Next(), IsDone(), CurrentItem()}   (soyut)
ConcreteAggregate {CreateIterator()} ──▷ Aggregate
   CreateIterator():  return new ConcreteIterator(this)
ConcreteIterator ──▷ Iterator
ConcreteAggregate ┈┈► ConcreteIterator            (oluşturur)
ConcreteIterator ──► ConcreteAggregate            (dolaştığı bileşiği bilir)
```

Bileşenler (slayttan):

- **Aggregate/ConcreteAggregate:** Parçaları arasında dolaşılacak nesnenin arayüzü/gerçeklemesi.
- **(Gezgin) Iterator/ConcreteIterator:** Dolaşma işleminin arayüzü/gerçeklemesi.
- `java.util` paketindeki veri yapıları bu kalıbı (daha basit biçimiyle) kullanır.

Bileşik nesneyi dolaşmak için `CreateIterator` metodu çağrılır; bu, `Iterator` türünden bir nesne döndürür. Iterator'da `First`, `Next`, `IsDone` gibi gereken dolaşım komutları kodlanır. Arka planda belirli bir somut yapı belirli bir somut dolaşım nesnesiyle ilişkilidir; istemcinin dolaylı olarak kullandığı odur. Farklı türden bileşik nesneler de aynı biçimde dolaşılabilir.

6.2 Gerçekleme kararları

Hoca, birçok kalıpta olduğu gibi burada da koda geçmeden bazı ayrıntılara karar vermek gerektiğini söyledi.

Dolaşılan parçanın durumu değiştirilebilecek mi? Iterator'la o anda dolaşılan bir nesnede setter metotları ya da nesnenin durumunu değiştirecek başka komutlar kullanılabilsin mi? İzin verilmemesi gerekiyorsa iterator, dolaştığı bileşiğin parçalarının kendilerini değil bir kopyasını döndürebilir. O zaman döndürülen nesne üzerinde yapılan değişiklik bileşik varlığın durumunu değiştirmez.

Dolaşma algoritmasını kim gerçekleyecek? Örneğin ikili ağaçta farklı dolaşma modları vardır. Hangisi kullanılacak ve bu kod aggregate'e mi, iterator'a mı yazılacak? (Üst arayüzler soyut sınıf olsaydı oraya da bir şeyler konabilirdi; ama arayüz yetiyorsa arayüz kullanmak daha doğru bir seçimdir.)

Dolaşılan nesneler istekçiye nasıl verilecek? Iterator'daki `First`, `Next` gibi metotlarla istemci parçaları tek tek alabilir. Ya da her seferinde sıradaki nesneyi döndürmek yerine, dolaşılmış bir dizi ya da bütün elemanların bir diziye kopyalanmış hâlinin başlangıç adresi döndürülebilir. Yani teker teker mi, hepsi birden mi?

Hoca bunun iki ayrı soru olduğunu belirtti: dolaşma algoritmasını kimin gerçekleyeceği ve dolaşılan nesnelerin nasıl gönderileceği. Komut verme sırasını istekçi de belirleyebilir, gezgin de. Farklı dolaşma yöntemleri (LDR, DLR vb.) varsa istemci "şu biçimde dolaşmanı tercih ediyorum" diyebilmelidir. Bu, iterator'ın bir enum üyesini ayarlamakla olabilir; ya da istek aggregate'e verilir, o da uygun iterator'ı döndürür. Farklı gerçeklemelerden hangisinin probleminize uygun olduğunu anlayıp ilgili kodu ilgili tarafa eklemek gerekir.

7. Iterator Uygulama Örneği: İki Yemek Firmasının Menüleri

7.1 Problem

Örnek Head First Design Patterns'tan doğrudan alınmıştır. Online yemek siparişi işine girmek istiyorsunuz (pandemi döneminde paket servisin önemi de arttı). Kendi lokantanızı kurmak yerine iki yemek firması satın aldınız: biri kahvaltılıkta (Pancake House), öteki yemekte (Diner) uzman. Menülerini halka açıp sipariş almak istiyorsunuz.

Sorun şu: her firma kendi menü gerçeklemesini yapmış. Menüdeki elemanları listeleme gibi metotlar tesadüfen aynı olsa bile (olmasa basit bir adapter konurdu), bir firma menüyü dizi, öteki liste kullanarak gerçeklemiştir. Bunları ortak kod üzerinden müşteriye nasıl sunarız? Ortak kod şarttır, çünkü ileride başka firmalar da satın alınacaktır. "Şu firmanın menüsünü göstereceksem şunu yap, kahvaltılık firmasınınkini göstereceksem bunu yap" diye kendimi bir `if-else-else` zincirinde bulmak istemiyorum.

> **Hocanın vurgusu:** Böyle bir `if-else` zinciri kötü tasarımın en önemli göstergelerinden biridir.

Ayrıca vejetaryenler de düşünülmek isteniyor; satın alınan firmaların da ayrı vejetaryen menüleri var. Böylece hem yemek türü (kahvaltılık mı, yemek mi) hem de vejetaryen olup olmaması şeklinde iki parametre ve dört seçenek ortaya çıkıyor.

7.2 Mevcut kod ve hızlı-kirli çözüm (ekrandan, 01:00:00)

İki firma da ortak bir `MenuItem` sınıfı kullanıyor: ad, açıklama, vejetaryen olup olmadığı (`true` ise vejetaryen) ve fiyat. Vejetaryen olanlarda örneğin pankekte sosis olmaz, yumurta ya da yaban mersini olabilir.

- `PancakeHouseMenu` menüyü bir `ArrayList` ile tutar; yapıcıda menü öğelerini oluşturur, menü öğesi ekleme metodu vardır ve `getMenuItems()` ile bütün `ArrayList`'i döndürür.
- `DinerMenu` menüyü bir dizi ile tutar; dizilerle çalışıldığı için eleman ekleme kodu biraz daha uzundur ve `getMenuItems()` dizi döndürür.

Biri liste, biri dizi döndürüyor. Hızlı ve kirli çözüm (`WaitressQuickAndDirty`): garson önce kahvaltılıkları listeler, kahvaltı firmasının menü öğelerini alır ve bir liste döngüsüyle dolaşır; sonra öğle yemeklerini listeler, yemek firmasının öğelerini alır ve bu kez dizi döngüsüyle dolaşır. Ekranda gösterilen vejetaryen menü kodunun ilgili kısmı:

```
ArrayList<MenuItem> breakfastItems = pancakeHouseMenu.getMenuItems();
for( int i=0; i<breakfastItems.size(); i++ ) {
    MenuItem item = breakfastItems.get(i);
    if( item.isVegetarian() )
        printMenuItem(item);
}
System.out.println("\nLUNCH");
MenuItem[] dinerItems = dinerMenu.getMenuItems();
for( int i=0; i<dinerItems.length; i++ ) {
    MenuItem item = dinerItems[i];
    if( item.isVegetarian() )
        printMenuItem(item);
}
```

Menü öğesini yazdırma (adı, fiyatı, açıklaması) ortak bir `printMenuItem` metoduna konmuştur; aynı kod her yere ayrı ayrı yazılsaydı daha büyük bir bakım kabusu çıkardı. Yine de bu hızlı

çözüm, iş genişledikçe (öğle yemeği seçenekleri, yeni firmalar, yeni özellikler) bakımı çok daha zor bir koda dönüşür.

7.3 Iterator kalıbı ile çözüm (ekrandan, 00:50:00)

```
«interface» Menu
   + createIterator(): Iterator<MenuItem>
«interface» Iterator<MenuItem>
   + hasNext(): boolean
   + next(): MenuItem

Waitress ──► Menu
Waitress ┄┄► Iterator<MenuItem>
DinerMenu ┄┄▷ Menu                  PancakeHouseMenu ┄┄▷ Menu
DinerMenuIterator ┄┄▷ Iterator      PancakeHouseMenuIterator ┄┄▷ Iterator
DinerMenu ┄┄► DinerMenuIterator,    DinerMenuIterator ──► DinerMenu
PancakeHouseMenu ┄┄► PancakeHouseMenuIterator,    PancakeHouseMenuIterator ──► PancakeHouseMenu
```

Garson (`Waitress`) bir `Menu` arayüzü üzerinden hem kahvaltı hem yemek firmasının menüsüne erişir. Menünün ayrıntısını, dizi mi liste mi kullandığını bilmesine gerek yoktur; yalnız menüden bir iterator isteyebilmesi yeter. Iterator'ın döndürdüğü `MenuItem`'lar üzerinde, "sıradaki eleman var mı, varsa döndür" komutlarını çağırarak çalışır.

Bunu gerçeklemek için `DinerMenu`'ye bir iterator döndüren `createIterator` metodu eklenir. Dönen tür Java'nın kendi `java.util.Iterator` arayüzüdür; hoca ayrı bir iterator arayüzü kodlamaya gerek görmemiş. Ama Java'nın iterator'ı, `MenuItem` türünden sıradaki elemanın nasıl verileceğini ve sonraki elemanın olup olmadığını bilemez; bu yüzden `DinerMenuIterator` sınıfı kodlanmıştır (örnek kitaptan alınmıştır):

- Dizi kullanıldığı için şu anki konum tutulur.
- `hasNext`: dizinin sonuna ulaşılmışsa `false`, aksi takdirde `true` döndürür.
- `next`: o konumdaki elemanı döndürür ve konumu bir ilerletir.

Kahvaltılık menüsüne de iterator döndüren bir metot eklenmiş, `PancakeHouseMenuIterator` benzer biçimde ama `ArrayList`'ten elemanı alarak çalışır. İki iterator karşılaştırıldığında çok fark yoktur; ikisi de konumu tutar ve o konumdaki elemanı döndürür, `ArrayList` sayesinde `hasNext` biraz daha basittir. `remove` işlemine de izin verilmiştir: listenin avantajı burada görülür, çünkü listenin ortasından eleman silmek kolaydır, dizinin ortasından silmek ise sonraki bütün elemanları bir sola kaydırmayı gerektirir.

Esas avantaj garsonun kodundadır. `DinerMenu` ve `PancakeHouseMenu` bir `Menu` arayüzü altında, iterator'lar da ortak `Iterator` arayüzü altında toplanmıştır; garson farklı firmaların menülerine yalnız arayüz üzerinden erişir. Böylece kod bağımlılığı azalır. Garson yapıcıda iki menüyle ilklendirilir; menüyü yazdırırken önce kahvaltı menüsünün iterator'ını alıp başlığı yazar ve ortak `printMenu(Iterator)` metodunu çalıştırır, sonra yemek firmasının iterator'ını alıp aynı metodu çağırır. `printMenu` herhangi bir firmanın menüsü için aynıdır: iterator'da sıradaki eleman varsa ona geç ve yazdır.

Vejetaryen menü için kalıplı sürüm (ekrandan, 01:00:00):

```java
public void printVegetarianMenu() {
    System.out.println("\nVEGETARIAN MENU\n----\nBREAKFAST");
    printVegetarianMenu(pancakeHouseMenu.createIterator());
    System.out.println("\nLUNCH");
    printVegetarianMenu(dinerMenu.createIterator());
}

private void printVegetarianMenu(Iterator<MenuItem> iterator) {
    while (iterator.hasNext()) {
        MenuItem menuItem = iterator.next();
        if (menuItem.isVegetarian()) {
            printMenuItem(menuItem);
        }
```

```
    }
}
```

Yalnız kahvaltılıklar mı yazdırılmak isteniyor? Yalnız kahvaltı firmasından iterator alınır. Yalnız yemek menüsü mü? Yalnız yemek firmasından iterator alınır. Bu ek zenginliği hızlı-kirli sürüme eklemek, tek tek elemanlar içinde dolaşmayı her yerde yeniden yazmayı gerektirdiği için daha karmaşıktır.

7.4 Karşılaştırma ve değerlendirme

Hoca iki garson sınıfını Eclipse'in karşılaştırma aracıyla yan yana gösterdi: `printMenu` basitleşmiş, kısalmış ve odaklanmış; `printMenuItem` aynı kalmış; `printVegetarianMenu`'de ufak bir bölümleme yapılmış (dolaşma mantığı iterator parametresi alan ayrı bir metoda taşınmış). Program çalıştırıldığında iki sürüm de aynı çıktıyı (kahvaltı ve öğle menüsü) verir. Fark performansta ya da müşterinin gördüğünde değildir; farklı bir seçenek eklemek kalıplı sürümde daha kolay olacaktır.

Hoca bu örnekte iterator kalıbını kurmak için harcanan çabanın, kodu çekidüzene sokmak için zaten harcanacak çabaya denk düştüğünü, bu yüzden değdiğini söyledi. Her zaman böyle olmaz: eldeki bir çözümü bir tasarım kalıbıyla düzeltmek bazen, kalıpsız ama derli toplu bir çözüm yapmaktan daha zor olabilir. Ama arada fark az olduğunda kalıbı kullanarak daha doğru bir iş yapılır.

7.5 Soru: Iterator ile Adapter'ın farkı (derste sözlü, 01:02:35)

Soru: Iterator, Adapter'a çok benziyor gibi; burada da yeni bir iterator sınıfı oluşturup farklı yapıları ortak metotlarla çalıştırıyoruz. Yanlış mıyım?

Cevap: Amaçlar çok farklıdır. Adapter yalnız belli bir metot çağrısı içindir: belli bir türden, belli bir sınıfın belli bir metodunun çağrılması gerekir, aynı işi yapan kod ise farklı bir sınıfın farklı bir metodundadır; Adapter araya bir köprü koyar. Iterator ise kompozit bir varlığın her bir bileşeni arasında dolaşmayı sağlar ve bunu eş biçimde yapar. Her bileşenin belki farklı olan metotlarına ortak bir dolaşma arayüzü üzerinden eş biçimde erişilmesini sağlaması açısından Adapter ile ortaklığı vardır; ama Iterator yalnız eş biçimliliği değil dolaşımı da sağlar.

Soru (devam): Adapter ile de bu işi yapabilirdik: her yapı için bir adapter yazıp sonuçta hepsinden bir `ArrayList` döndürebilirdik.

Cevap: Bu örneğin odak noktası yalnız "farklı firmaların menülerini ortak bir yapı altında toplamak", yani farklı gerçekleme ayrıntılarını eş biçimli yapmak olsaydı, problem öyle sunulabilirdi. Ama burada esas gösterilen dolaşımdır. Farklı kompozit varlıkların birlikte eş biçimde dolaşılması gereken bir soruda Iterator kalıbını mutlaka anmak gerekir. Anılmasa bile, örneğin Java kullanılacağı önceden kararlaştırılmışsa, "yalnız adapter kullanırız, gerisini Java'nın iterator'ı halleder" diye en azından bu vurgulanmalıdır.

8. Memento Kalıbı: Amaç ve Motivasyon

8.1 Adın kaynağı

Hoca kalıba "Memento" filmini sorarak girdi. Filmde karısının öldürülmesinin intikamını almaya çalışan bir adam vardır; adamın değişik bir hafıza bozukluğu vardır: uzak geçmişi hatırlar ama yakın geçmişi hatırlayamaz, bu yüzden kendine küçük notlar yazar. Memento bu anlamda küçük notlar, bir anı dondurmak, o anın bütün resmini gösteren bir hatıra, bir fotoğraf karesidir. Kalıp adını filmden değil, kelimenin bu anlamından alır: sistemin o anki durumunun bir hatırası.

8.2 Amaç (ekrandan, 01:20:00)

Slayttaki amaç:

- Bir nesnenin durum bilgisini sarmalama (encapsulation) ilkesini bozmadan elde etmek.
- Böylece nesnelerin ikincil saklama ortamlarında saklanması, geri yüklenmesi, ağ üzerinden aktarımı mümkün olabilir.

- Yine bu şekilde son yapılan işlemi geri alma düzeneği (undo) kurulabilir.
- Durum bilgisinin yalnız bir kısmının saklanması da yeterli olabilir.

Bir nesnenin durumunu almak için çeşitli gerekçeler olabilir: nesnenin durumunu bütünüyle diske saklamak, ya da Java'nın serialization/deserialization yeteneğinde olduğu gibi nesneyi bir bayt dizisine çevirip diske yazmak veya bir TCP bağlantısı üzerinden başka bir bilgisayara göndermek. Karşı tarafta ya da diskten geri yüklerken baytlardan nesne yeniden hayata geçirilir (deserialization).

8.3 Sorun: private üyeler ve sahtecilik

Ama nesnenin private üyeleri de vardır. Private üyelere açık erişim vermeden bu iş nasıl yapılır? Ayrıca saklama ve geri yükleme işini yapan nesne her kimse, onun sahtecilik yapmadığından, durumu değiştirerek saklamadığından nasıl emin olunur?

Bütün üyeler public yapılırsa, ya da saklayıcı aynı pakete konup paket erişimiyle, ya da kalıtım ilişkisiyle bütün üyelere erişirse, saklayıcı getter'larla her şeyi alır ama bazı şeyleri değiştirebilir de. Hem sahtekârlıktan kaçınmak hem de sarmalama ilkesine sadık kalarak nesnenin durum bilgisini saklamak istiyoruz.

Hocanın örneği: bir banka hesabı para yatırma ve çekme işlemleriyle farklı durumlardan geçer. Bir fatura ödenip bakiye düşülmemiş bir durum saklanırsa, daha sonra geri yüklendiğinde "havadan para kazanmış" olunur. Banka böyle bir şeyi istemez. Durumu farklı anlarda saklamanın bir başka nedeni de undo/redo işlemleridir; bazı durumlarda durum bilgisinin tamamıyla değil yalnız bir kısmıyla ilgilenilebilir.

8.4 Neden kalıtım ya da her sınıfa ayrı kod değil?

Bir nesne kendi önceki durumlarını kendisi saklayabilir; ama undo/redo yapılacak her tür varlık için bunu ayrı ayrı mı kodlayacağız? Hayır. Peki bu işlemler için özel metotlar içeren bir sınıf yazıp varlıklarımızı ondan mı türeteceğiz? Olabilir, ama Java gibi dillerde yalnız bir kalıtım hakkınız vardır ve bu değerli hakkı yalnız undo/redo için harcamak istemeyiz; varlık iş mantığı gereği başka bir kalıtım ilişkisi içinde olabilir. Bütün bunlar Memento kalıbına götürür.

9. Memento Kalıbının Yapısı ve Gizlilik Sorunu

9.1 Kalıp yapısı (ekrandan, 01:30:00)

```
Originator {SetMemento(Memento m), CreateMemento(); state} ┄┄► Memento {GetState(), SetState()
Caretaker ◇──memento──► Memento
   CreateMemento():  return new Memento(state)
   SetMemento(m):    state = m->GetState()
```

Bileşenler (slayttan):

- **Originator:** Durum bilgisi saklanacak olan nesne. Mementoları oluşturan odur.
- **Memento:** Durum bilgisinin gerekli kısımlarını saklar.
- **Caretaker:** Gerekli anlarda durum bilgisinin bir kopyasını alır ve elindeki kopyalardan birini geçerli durum hâline getirir.

Caretaker, Originator'dan aldığı mementoları saklar; günce tutuyor olabilir ya da yukarıdaki senaryoları gerçekleştiriyor olabilir. Aynı Originator'dan alınmış birden fazla mementoya sahip olabilir (ör. undo/redo için). Akış şöyledir: Caretaker Originator'dan zaman zaman memento ister. Originator `CreateMemento` ile yeni bir memento oluşturur, o anki durum bilgisinin saklanması gereken kısmını ona aktarır ve mementoyu döndürür. Bir süre sonra, ör. bir undo isteği geldiğinde, Caretaker Originator'a bir memento verip "senin bir önceki hâlin bu, buna geri dön" der; Originator `SetMemento` ile parametre olarak aldığı mementodan durum bilgisini okuyup kendi durumunu o hâle getirir.

9.2 Kullanım ve kritik gerçekleme ayrıntısı

Kalıp, nesnenin durum bilgisinin tamamının ya da bir kısmının, sonradan o duruma geri dönebilmek amacıyla saklanması gerektiğinde kullanılır. Bu başka yollarla da yapılabilir; ama

gizlilik ilkesini bozmadan yapılmak isteniyorsa Memento kalıbı kurulur.

> **Hocanın vurgusu:** Memento'nun metotlarına yalnız Originator erişebilmelidir.

Memento'nun `GetState` ve `SetState` metotları vardır. `SetState`'i yalnız Originator çağırabilmeli, Caretaker çağıramamalıdır. Caretaker yalnız mementoları alıp bakımını yapmalı: saklamalı, undo/redo isteği gelince "şu hâle geri dön, şu hâle ileri git" diyebilmeli, gerekirse mementoları saklayıp isteyene verebilmelidir. `GetState`'i belki çağırmak isteyebilir ama bunun için de çok neden yoktur. Şemada Memento'nun yapıcısı da yoktur: Caretaker sıfırdan memento da üretememelidir. Gizlilik sıkıca korunacak ve sahte memento oluşturulması engellenecekse bu gerçeklenmek zorundadır.

9.3 Dile göre çözümler

GoF kitabı Java'dan söz etmez; hoca bunun haklı olduğunu, çünkü farklı nesneye dayalı dillerde farklı mekanizmalar bulunduğunu söyledi (Java, GoF kitabından sonra çıkmış bir dildir).

- **C++:** Memento'nun metotları private yapılır ve Originator, Memento'nun `friend`'i yapılır. Sınıflar arasındaki `friend` ilişkisiyle gizlilik korunur.
- **Java, paket çözümü:** Memento ve Originator aynı pakete konur, Caretaker başka bir pakette olur. Memento'nun `getState`/`setState` metotları paket erişimli olduğu için Caretaker paketin dışından erişemez, Originator erişir. Ancak bu durumda Originator ile aynı pakete konan başka sınıflar da Memento'yu bozabilir. Bunu önlemek için paket `sealed` olarak işaretlenir: paket bir jar dosyası olarak müşterilere gönderilirken `sealed=true` ile oluşturulursa o pakete sonradan sınıf eklenemez. Böylece Memento'ya erişmek isteyen birinin basitçe aynı pakete bir sınıf eklemesi engellenir.
- **Hocanın önerisi, iç sınıf:** Hoca paket çözümünü pek sevmediğini söyledi. Memento, Originator'ın iç sınıfı yapılırsa, herhangi bir Caretaker (ister aynı pakette, ister farklı pakette, ister kalıtım ya da başka bir ilişki içinde olsun) Memento'nun metotlarına erişemez.

  Ek bilgi (derste anlatılmadı): Java'da bir iç (nested) sınıfın private üyelerine onu kapsayan dış sınıf erişebilir; dış sınıfın private üyelerine de iç sınıf erişebilir. Bu kural, iç sınıf çözümünde Originator'ın Memento'nun private `getState`/`setState` metotlarını çağırabilmesini, dışarıdaki Caretaker'ın ise çağıramamasını sağlar.

10. Memento: İç Sınıf ile Çözüm ve Derste Denenen Açık

10.1 Kod (ekrandan, 01:38:40 ve 01:50:00)

Saklanması gereken durum bilgisi bütünüyle bir `State` nesnesinde sarmalanmıştır (başka üye alanlar da eklenebilirdi). `State` burada basitçe bir metin (durum) tutar; örnek genişletilmek istenirse durum bilgisine yapılacak eklemeler bu sınıfa yapılmalıdır. `State`'in getter ve setter metotları vardır ve `Cloneable`'ı gerçekler; kendi `clone` metodu kendisinin aynı duruma sahip bir kopyasını oluşturur. Hoca JDK ile gelen `Object.clone`'a bel bağlamak yerine `clone`'u kendisi kodlamayı tercih etti, çünkü klonlamada derin kopya (deep copy) ve sığ kopya (shallow copy) ayrımı vardır ve hangisinin yapılacağına kendisi karar vermek istedi.

```
package dp.mementoV2.innerClass;
public class Originator {
   private State state;
   public Originator( ) { state = new State( ); }
   public Memento createMemento( ) {
      Memento m = new Memento( );
      m.setState(state.clone());
      return m;
   }
   public void setMemento( Memento m ) { state = m.getState(); }
   public void modify( String str ) { state.setDurum(str); }
   public String toString() { return state.getDurum(); }

   public class Memento {
      private State state;
```

```
        private State getState() { return state; }
        private void setState(State state) { this.state = state; }
    }
}
```

- Yapıcı yeni ve boş bir `State` oluşturur. Hoca, mementoya kendi oluşturacağı bir durumu verebilme ihtimali nedeniyle boş yapıcıyı tercih etmiş olabileceğini söyledi; Originator, iş mantığına uygun diğer metotlarla zaten istenen biçimde değiştirilebilir.
- `createMemento` boş bir memento oluşturur, durumunu Originator'ın o anki durumunun kopyası olarak atar ve mementoyu döndürür.
- `setMemento` parametre olarak aldığı mementonun durumunu alıp kendi durumu yapar.
- `modify`, Originator'ın durumunu değiştirmek içindir (bir tür yapıcı gibi düşünülebilir); `toString` varlığın o anki durumunun açıklamasını döndürür.
- Memento iç sınıftır ve içinde bir `state` ile onun private getter/setter'ı vardır. İç sınıfın kendisini private yapmaya gerek yoktur; public bırakılsa da istek yerine gelir.
- Hoca, klon kullanılmadığında izin verilen bir durum değişikliğinin mementoyu da etkilediğini, `Cloneable` yapınca bunun düzeldiğini hatırladı.

Caretaker aynı zamanda örneğin test programıdır:

```
package dp.mementoV2Outer;
import dp.mementoV2.innerClass.*;
import dp.mementoV2.innerClass.Originator.Memento;

public class Caretaker {
   public void test( ) {
      Originator subject = new Originator( );
      subject.modify("ilk durum");
      Memento mem = subject.createMemento( );
      subject.modify("yeni durum");
      //bir süre sonra...
      subject.setMemento( mem );
      System.out.println(subject);
   }
   public static void main( String[] args ) {
      Caretaker ct = new Caretaker();
      ct.test();
   }
}
```

Bir Originator oluşturulur ve durumu "ilk durum" yapılır; bir memento istenir; durum "yeni durum" yapılır; bir süre sonra eski mementoya geri dönülür ve durum yazdırılır. Beklenen çıktı "ilk durum"dur. Hoca ara adımları da yazdırarak çalıştırdı: çıktı sırasıyla "ilk durum", "yeni durum", "ilk durum" oldu.

Caretaker başka bir paketteyken Memento'nun metotları private olduğu için onlara erişemez. Ayrıca iç sınıftan nesne oluşturmak için mutlaka dış sınıftan bir nesne gerekir ve varsayılan yapıcı da eklenmez; Caretaker istediği gibi boş bir memento oluşturamaz.

10.2 Sahte memento denemesi (derste sözlü, 01:41:59)

Hoca, Caretaker'ın araya "olmaz durum" gibi sahte bir durum sokup sokamayacağını canlı denedi. Doğrudan `new Memento()` denemesi derlenmez. Ama şu yol işe yaradı (ekrandaki açıklama notu, 01:50:00):

```
Originator fakeSubject = new Originator( );
fakeSubject.modify("sahte durum");
Memento memFake = fakeSubject.createMemento();

subject.setMemento( memFake );
```

Caretaker kendi sahte Originator'ını oluşturup istediği duruma getirir, ondan meşru yoldan bir memento alır ve bunu asıl nesneye yükler. Böylece asıl nesne "sahte durum"a geçer. Hoca

bunu, private üyelerin aynı sınıftan bütün nesneler için erişilebilir olmasıyla (private erişimi nesne değil sınıf düzeyindedir) açıkladı ve dilin temel tasarım kuralını değiştiremeyeceği için bunu doğrudan engelleyemeyeceğini söyledi. Bu senaryo küçük bir Java denemesiyle de doğrulandı: `setMemento(memFake)` sonrasında `subject` "sahte durum" yazdırır.

Önerilen önlemler:

- **Kimlik denetimi:** Her Originator nesnesine bir evrensel tekil kimlik (UUID) verilir ve bu kimlik mementonun içine konur. `setMemento` sırasında mementoyu oluşturan Originator'ın kimliği kendi kimliğiyle aynı değilse işlem yapılmaz. Hoca, kimlik private olsa bile ele geçirilip geçirilemeyeceğinin ayrıca düşünülmesi gerektiğini ve bunu sonra düşünmek üzere not aldığını söyledi.
- **Bakımı Originator'a vermek:** Mementoların bakımı Caretaker yerine Originator'ın kendisine kodlanır. Originator bir memento listesi tutar; her yeni `createMemento` kendini bu listeye ekler. Caretaker artık memento vermez, yalnız `setMemento` yerine `activateMemento(int ...)` gibi bir metotla "şu kadar önceki hâline dön" ya da "şu numaralı hâle dön" der. Böylece dışarıdan memento verilmesine gerek kalmaz ve sorun çözülür. Ancak mementolara mutlaka ayrı bir Caretaker bakacaksa ve araya sahte memento konamasın isteniyorsa daha ayrıntılı düşünmek gerekir; hoca bunun belki mümkün olmadığını, belki de bilmediği bir dil özelliğiyle yapılabileceğini söyledi.

10.3 Soru: Create and set memento (derste sözlü, 01:46:26)

Soru: `createMemento` metodunun içinde `setMemento` da yapsak, yani mementoyu oluşturduğumuz anda setlesek ("create and set memento" gibi tek bir metot), dışarıdan memento verilemez; `setMemento`'nun public olmasına da gerek kalmaz. Olur mu?

Cevap: Dışarıdan memento almak gerekecektir, çünkü belli bir anda alınmış bir mementoya, yani o duruma geri dönmek gerekir. Originator açısından mantık şudur: "Caretaker'a güveniyorum; sen beni bir durumdan almışsın, bana yine benden aldığın durumlardan birini geri ver." Bu yüzden ayrı bir set metoduna ihtiyaç vardır. Bu tartışma, hocanın yukarıdaki "bakımı Originator'a ver, Caretaker numarayla istesin" önerisine götürdü.

11. Memento Uygulama Örneği: Doküman Undo/Redo

11.1 Şema (ekrandan, 01:40:00)

```
Controller 1 ──► * Document
Document 1 ──► * DocumentMemento
Not: Controller'ın DocumentMemento ile hiçbir şekilde ilişkisi yok.

Document
   - currentState: DocumentMemento
   - states: List<DocumentMemento>
   - position: int
   + undo()
   + redo()
   + anyOperation()

DocumentMemento
   - documentState
```

```
public void undo() {
 position -= 1;
 currentState = states.get(position);
}
public void redo() {
 position += 1;
 currentState = states.get(position);
}
public void anyOperation() {
```

```
 states.add(currentState);
}
```

11.2 Değerlendirme

Undo/redo olacaksa, bir pozisyon geri git, bir pozisyon ileri git; herhangi bir işlem yapıldığında da mevcut mementolar arasına yenisini ekle türünden bir yapı doküman içinde kurulabilir. Controller yalnız undo ya da redo der. Kalıptaki karşılıklar: Document hem Originator hem Caretaker'dır; Controller uygulamadır; mementolar `DocumentMemento`'dur.

Hoca bunu "yarım memento" uygulaması olarak niteledi: Caretaker ayrı olmadığı sürece kalıp yarım uygulanmış olur. Arada bir ödünleşim (trade-off) vardır. Ama bu durum açıkça belirtilerek (ör. bir notta) bırakılırsa bu da kabul edilebilir bir çözümdür.

12. Observer Kalıbı: Amaç ve Yapı

12.1 Amaç

Gözlenen (observe edilen) bir nesne vardır. Bu nesnenin durumu değiştiğinde onunla ilişkili bütün diğer nesnelerin durumdan haberdar edilmesi ve güncellenmesi istenir. Yani bir gözleyen-gözlenen ilişkisi vardır; ama istenen çalışma biçimi şudur: gözleyiciler gözleneni sürekli incelemesin, gözlenen değiştiğinde gözleyicileri haberdar etsin. Ayrıca herhangi bir gözleyicinin herhangi bir gözleneni gözleyebilmesi ve bu ilişkilerin çalışma anında da kurulabilmesi istenir.

12.2 Örnek: hesap tablosu ve grafikler

Bu örnek MVC konusunda da verilmişti; burada işin aslının farklı olduğu görülecek. Hesap tablosundaki bilgi değiştiğinde değişikliklerin farklı çizelgelere de anında yansıtılması istenir. Arka planda bir veri vardır; bu veri Excel tablosuna girilmiş, ondan sütun ve pasta grafikleri çizilmiştir. Herhangi bir hücre değiştiğinde grafiklerin otomatik güncellenmesi istenir. Bu grafikler verideki değişiklikten nasıl haberdar olacak? Ayrıca yeni bir grafik ekleyebilmeliyiz; hatta bir Excel dosyasındaki veriden başka bir Excel dosyasında grafik tanımlanabiliyor.

12.3 Kalıp yapısı (ekrandan, 02:10:00 ve 02:40:00)

```
Subject {Attach(Observer), Detach(Observer), Notify()}        (soyut)
   Notify():  for all o in observers { o->Update() }
Subject ──observers──►* Observer {Update()}                   (soyut)
ConcreteSubject {GetState(), SetState(); subjectState} ──▷ Subject
   GetState():  return subjectState
ConcreteObserver {Update(); observerState} ──▷ Observer
   Update():  observerState = subject->GetState()
ConcreteObserver ──subject──► ConcreteSubject
```

Gözlenen nesne Subject, onu gözleyenler Observer'dır. Subject'te yapılacak gözlenme ilişkisine yönelik işler bir soyut sınıfta tanımlanmıştır. Subject'in değiştiğini Observer'a bildirebilmesi için Observer'ın bir `Update` metodu vardır.

Bir Subject birden fazla Observer ile ilişkili olabilir; bu yüzden Subject bir Observer listesi (ya da başka bir veri yapısı) tutar. Bu ilişkiyi yöneten metotlar soyut Subject sınıfındadır: gözleyici eklemek (`Attach`), çıkarmak (`Detach`) ve `Notify`. `Notify`, Subject'in durum bilgisinin değiştiğini bütün gözleyicilerine bildirir: listedeki her eleman için `Update` çağrılır. Bu, kalıbın temel işlerinden biridir. Subject'in alt sınıfları somut Subject'ler, Observer'ın alt sınıfları somut Observer'lardır.

Slayttaki bileşen açıklamaları:

- **ConcreteSubject:** Subject gerçeklemeleri. Durumu değiştiğinde gözleyicilerini Observer arayüzünü kullanarak bilgilendirir.
- **ConcreteObserver:** Observer gerçeklemeleri. Gözlediği nesneyi gösteren bir işaretçi tutabilir.

12.4 Somut gözleyici ile somut gözlenen arasındaki ilişki

Hoca slayta çizerek ikinci bir somut gözleyici (`ConcreteObserver2`) ve ikinci bir somut gözlenen (`ConcreteSubject2`) ekledi. Şemadaki `subject` ilişkisi, belli bir somut gözleyicinin belli bir somut gözleneni bilmesini sağlar. İki seçenek vardır:

- Her tür somut gözleyicinin her tür somut gözleneni gözleyebilmesi isteniyorsa, somut sınıflar arasındaki bu ilişki kaldırılıp soyut Observer'dan soyut Subject'e doğru kurulur.
- `ConcreteObserver` yalnız `ConcreteSubject`'i, `ConcreteObserver2` yalnız `ConcreteSubject2`'yi gözlesin isteniyorsa, somut gözleyiciden somut gözlenene olan özgül sahiplik ilişkisi tutulur.

Hangisi işe yarıyorsa o seçilir.

12.5 Subject neden soyut sınıf?

Subject, durumundaki değişiklikler izlenecek nesnenin arayüzüdür. Birden fazla gözleyicisi olabileceği için attach/detach/notify metotlarını koymak için en iyi yer burasıdır; bu yüzden Subject bir arayüz olamaz, soyut sınıf olması en mantıklısıdır. Observer bir arayüz olabilir. Ama gözleyici de gözlediği gözlenenleri tutacaksa ona da `attachSubject`, `detachSubject` gibi metotlar eklenir ve o da arayüz olmaktan çıkıp soyut sınıfa döner. İhtiyaca göre karar verilir.

13. Observer Gerçekleme Ayrıntıları

13.1 Pull model ve push model (ekrandan, 02:13:00)

Hoca bu kalıbı kurarken önce iki çalışma biçiminden birinin seçilmesi gerektiğini söyledi. İkisi de durum bilgisinin gözleyicilere nasıl gönderileceğiyle ilgilidir.

Pull model (slayttan):

- (+) Gözlenen yalnız gözleyicileri haberdar eder; gözleyiciler ilgili durum bilgisini gözlenenden alır.
- (+) Gözlenenin gözleyicilerle bağlantısını azaltan bir çalışma biçimidir.
- (+) Gözleyici durum bilgisinin yalnız istediği kısmını almayı seçebilir.
- (−) Gözleyici gözlenende nelerin değiştiğini kendisi çıkarmak zorunda kalır.

Bağlantının azalması, özellikle bir somut gözlenen farklı türden somut gözleyicilerce izlenebilecekse istenir; nesneye dayalı tasarımda düşük bağlaşım zaten istenen bir durumdur. Bir somut gözleyici durum bilgisinin bir kısmıyla, başka bir somut gözleyici başka bir kısmıyla ilgileniyor olabilir. Haberleşme masrafı da azalır: gözlenen yalnız "ben değiştim" diyen basit bir ileti gönderir, gözleyiciler istedikleri getter'ları çağırarak yalnız ilgilendikleri kısmı çeker. Nesne megabaytlarca veri tutuyorsa tüm durumu çekip kotayı yemek yerine yalnız ilgili kısmı çekmek daha az haberleşme kotası harcar.

Eksi yön için hoca bir benzetme yaptı: gözlenen yalnız "ben değiştim" der, ama neyin değiştiğini söylemez; tıpkı "bende bir değişiklik fark etmedin mi?" sorusunda saçın mı, makyajın mı değiştiğini kendinizin çıkarmak zorunda kalmanız gibi. Bunun için gözleyiciye ayrı kod yazmak gerekir; hoca, belki önceki durumu saklamak için Memento'nun da işe karışabileceğini, ama Memento'suz da yapılabileceğini söyledi.

Push model (slayttan): Gözlenen, gözleyicilere durum değişikliği hakkında ayrıntılı bilgi gönderir.

Genelde gözlenen kendi durum bilgisinin tümünü gönderir. Bir gözlenen birden fazla gözleyici tarafından izleniyorsa aynı durum bilgisi birçok iletişim kanalından birçok gözleyiciye gider; haberleşme masrafı artar. Buna karşılık gözleyici kodu daha basittir, çünkü neyin değiştiğini kendisi çıkarmak zorunda kalmaz. Hatta gözleyicinin tek bir `update` metodu yerine "şuram değişti, buram değişti" diyen farklı `update` metotları olabilir; bu durumda gözlenen yalnız değişen bilgiyi gönderir ve haberleşme masrafı yine azalır, ama gözlenenin kodu karmaşıklaşır. Gözlenen kodlanırken, durumunu değiştirebilecek her metoda gözleyicileri uygun biçimde haberdar eden çağrılar da konmalıdır.

Slayttaki ek ayrıntı: Aynı türden gözlenen nesnelerin tümünün gözleyicilerinin ortak olması istenirse, gözlenen gözleyicilerini statik bir veri yapısı içinde tutabilir.

13.2 Diğer ayrıntılar (ekrandan, 02:20:00)

- Bir gözleyicinin birden fazla (hatta farklı türden) nesneyi gözlemesi istenebilir; bu çok dert değildir.
- Gözlenen nesnenin durumundaki farklı türden değişiklikleri farklı türden gözleyicilerin izlemesi istenebilir. Hocanın örneği Java'daki listener mekanizmasıdır: bir çerçevede (frame) farklı paneller ve farklı olaylar (fare olayları, klavye olayları) vardır. Bir panel bir klavye dinleyicisiyle ilişkilendirilirse, dinleyici yalnız klavyeyle ilgili olayları alır, diğerlerini geri bırakır ya da hiç çekmez. Bu durumda kalıp yapısını ayrıntılandırmak gerekebilir.

13.3 Haber verme mesajını kim tetikleyecek?

Tek ya da çok `update` metodu olsun, gözleyen-gözlenen ilişkileri nasıl olursa olsun, önemli bir soru kalır: bildirimi kim tetikleyecek?

Hocanın sözlü anlatımı:

- **Gözleyici tetiklerse:** Gözleyici belli aralıklarla gözlenene "değiştin mi?" diye sorar ve durumu çeker (bunun için ayrı bir "değiştin mi" metoduna gerek yoktur ama eklenebilir). Bu, başarımı ciddi etkileyebilir: gözlenen seyrek değişiyorsa sık sorgulamak verimsizdir; gözlenenin ne zaman değişeceği hiç belli değilse sık aralıklarla sorgulamak zorunlu olur ve her sorgu bant genişliği harcar. Avantajı, gözleneni kodlayanın rahat etmesidir: yalnız iş mantığını yazar, gözleyicileri bildirme kodunu yazmaz bile.
- **Gözlenen tetiklerse:** `Notify` herkesi birden bilgilendirirse, gözlenenin farklı kısımlarıyla ilgilenen bazı gözleyiciler, ilgilendikleri kısım değişmediği hâlde boşuna bilgilendirilmiş olur. Ya da durum değişir, gözleyiciler bilgilendirilir, çok kısa süre sonra yeniden değişir ve yeniden bilgilendirilir; belki bu kadar sık bildirim gerekmez. Gözlenen tarafında bir iyileştirme kodlanabilir: iki durum değişikliği arası çok kısaysa ilk değişikliği boşuna bildirmeyip yalnız son durumu bütün gözleyicilere göndermek.

Slayttaki metin (ekrandan, 02:20:00):

- Gözleyiciler tetiklerse, gözlenenlerin istekçilerini kodlayan kişi tetikleme komutunu vermeyi unutabilir.
- Gözlenen nesne tetiklerse, gözlenenlerin istekçilerinin peş peşe yapacağı durum değişiklikleri, tek bir haber verme mesajıyla gönderilebileceği yerde birçok izleyiciye gidecek bir sürü mesaj gönderilmesine neden olabilir.

  Ek bilgi (derste anlatılmadı): GoF kitabında bu soru şöyle konur: bildirimi ya gözlenenin durumu değiştiren metotları değişiklikten sonra `Notify` çağırarak tetikler, ya da gözlenenin istemcileri (istekçileri) uygun anda `Notify`'ı çağırmaktan sorumlu tutulur. İkinci yolun avantajı, istemcinin peş peşe birkaç değişiklikten sonra tek bir bildirim yapabilmesi; dezavantajı, istemcinin `Notify`'ı çağırmayı unutabilmesidir. Slayttaki "istekçileri kodlayan kişi tetikleme komutunu vermeyi unutabilir" ifadesi büyük olasılıkla bu ikinci yola karşılık gelir.

Hoca, kalıbı kurarken bütün bunların düşünülmesi gerektiğini vurguladı: önce pull ya da push modeline karar verilir, sonra bu ayrıntılar düşünülerek devam edilir.

13.4 Ne zaman kullanılır? (ekrandan, 02:20:00)

Slayttaki kullanım anları:

- Herhangi bir soyutlamanın birbirine bağımlı birden fazla parçası olduğunda.
- Bir nesnedeki değişikliklerin derleme anında bilinmeyen sayıda nesneyi de etkilemesi gereken anlarda.
- Bağlaşımı artırmadan bir nesnenin diğer nesneleri herhangi bir durumdan haberdar etmesi istenen anlarda.

Bağlaşım konusunda hoca, kalıbın bağlaşımı çok az artırdığını söyledi: tek bir çoklu sahiplik ilişkisi kurar, başka bir şey eklemez. Bir gözlenen farklı türden gözleyicilerin hepsi tarafından gözlenebilecekse, her tür gözleyici için ayrı tek yönlü sahiplik ilişkileri kurmak yerine soyutlama katmanı işi tek bir ilişkiyle, tek bir bağımlılıkla çözer. Birbirine bağımlı parçalarda

bağımlılığın yönüne göre değişiklikten etkilenen gözleyici, öteki gözlenen olur. Parçalar birbirini karşılıklı etkiliyorsa bir varlık hem gözleyici hem gözlenen olabilir; kurulum karmaşıklaşır ama yine yapılabilir.

> **Hocanın vurgusu:** Observer, az genel öğüt ama çok gerçekleme ayrıntısı veren kalıplardan biridir: pull model, push model, kısmi bildirim, tüm durumu aktarma gibi birçok ayrıntı vardır ve her biri farklı tasarımlarla farklı biçimlerde yapılabilir.

14. Observer Uygulama Örneği: Müşteri İlişkileri Programı

14.1 Problem ve birinci çözüm (ekrandan, 02:26:30)

Slayttaki problem: Yazdığımız bir müşteri ilişkileri programına yeni müşteri eklendiğinde, müşteriye bir "hoş geldiniz" e-postası gönderilecek ve müşterinin ikamet adresinin doğru olup olmadığına bakılacak.

Çözüm 1: Bu davranışlar sabit bir şekilde kodlanır.

```
Customer {+addCustomer()} ┈┈► WelcomeLetter       (generates Welcome Letters for customers)
Customer {+addCustomer()} ┈┈► AddrVerification    (verifies address of customers)
Not: addCustomer calls WelcomeLetter and AddrVerification
```

`Customer` sınıfının `addCustomer` metodu müşteriyi (ör. bir veritabanına) ekler. Müşteri bir `WelcomeLetter` ve bir `AddrVerification` nesnesiyle ilişkilendirilmiştir (bunlar ayrı servisler de olabilir). `addCustomer` içinde önce `WelcomeLetter`'ın ilgili metodu çağrılarak müşteriye hoş geldin e-postası gönderilir, sonra `AddrVerification`'ın ilgili metodu çağrılarak adresin tutarlılığı denetlenir. Bu yapılabilir, sorun değildir.

Peki ya gereksinimler değişirse? Yeni bir gereksinim çıkıyor: yeni müşteri şubelerimizden birinin 10 km yakınındaysa (10 km yarıçaplı bir daire içindeyse), göndereceğimiz mektup sıradan bir mektup olmasın, yalnız o mağazada geçerli bir indirim kuponu da içersin. Yazılımcı olarak benzer yeni isteklerin de çıkabileceğini düşünüp şimdiden bir altyapı kurmak isteyebiliriz: her yeni müşteri eklendiğinde bir ya da birkaç farklı işlem yapılabilsin ve sistem, müşteri eklendiğinde yapılması gereken işler olabileceğinden haberdar edilsin. Böyle düşününce Observer kalıbı akla gelir.

14.2 İkinci çözüm: Observer kalıbı (ekrandan, 02:30:00)

```
Customer
   - myObs: Vector<ICObserver> {static}
   + attach(ICObserver) {static}
   + detach(ICObserver) {static}
   + getState()
   + notifyObs()
   Not: notifyObs metodu gözleyicilerin update metodunu çağırarak onları haberdar eder.
Customer ──► «interface» ICObserver {+ update(myCust: Customer)}
AddrVerification ┈┈▷ ICObserver
BrickAndMortar ┈┈▷ ICObserver
WelcomeLetter ┈┈▷ ICObserver
```

Slayttaki not: Yalnız müşteri nesneleri izlenecekse, soyut bir 'Subject' üst sınıfı hazırlamaya gerek kalmadan Observer kalıbı kullanılabilir.

Tasarımın ayrıntıları:

- `Customer` sınıfına bir gözleyici listesi ve bir gözleyici arayüzü eklenir. `update` metoduna müşteri bilgilerinin tümü (müşteri nesnesi) verilir; yani bu örnekte push yaklaşımı kullanılır. Push yaparken genelde tüm durum bilgisi aktarılır, ama duruma göre kısmi bilgi de aktarılabilir; hoca, her yeni müşteri için yapılacak işleri şimdiden öngöremediği için bu karmaşıklığı sonraya bıraktığını söyledi.
- `attach` ve `detach` (ve liste) statik yapılabilir: her müşteri nesnesinin hep aynı gözleyicilerle izlenmesi isteniyorsa statik olabilir. Örnekte de böyle yapılmıştır; normal üye alan olmak zorunda değildir.

- `getState` metodu ve `notifyObs` metodu vardır. `notifyObs`, kalıbın gerektirdiği biçimde `myObs`'taki bütün gözleyicilerin `update` metodunu çağırarak müşterinin bilgisini onlara gönderir.
- İndirim kuponu işi `BrickAndMortar` adlı bir sınıfa konmuştur. E-ticaret yapanlar fiziksel mağazalarına "tuğla ve harç" (brick and mortar) adını verir; ad buradan gelir.
- `AddrVerification` gözleyici arayüzünü gerçekler; `update` metodunda müşterinin durumunu alıp gereken işleri yapar. Hoca kodun düz (straightforward) olduğunu söyledi.

Yeni bir kampanya ya da gereksinim geldiğinde, Observer arayüzünün yeni bir somut gerçeklemesi yazılır; durum bilgisi istenen biçimde alınarak yeni iş kodlanır, `Customer`'a dokunulmaz.

Gözleyicilerin statik tutulması üzerine hoca şunu ekledi: farklı tür müşteriler için farklı kampanyalar (belli bir gelir grubu ya da demografik grup için) yapılacaksa, ilgili kampanyanın kendi gözleyici sınıfı müşteri nesnesini sorgulayarak ona göre davranır. Ama sonuçta her yeni müşteri nesnesi aynı gözleyicilerin gözlemi altında kalacağından, bu örnekte gözleyicileri statik tutmanın hiçbir sakıncası yoktur.

15. UML Şemalarının Çizilmesi

Dersin sonunda hoca, derste güncel olarak çizdiği sınıf şemalarını Violet UML Editor ile çizdiğini gösterdi (ekrandan, 02:50:00). Yeni bir sınıf diyagramı oluşturulur; araç kutusundan sınıf seçilip tıklanarak sınıflar eklenir; "Inherits from" seçilip alt sınıftan üst sınıfa çizilerek kalıtım ilişkisi kurulur. Araç kutusunda "Implements interface", "Is associated with", "Is an aggregate of", "Is composed of", "Depends on" gibi ilişkiler ve not ekleme de bulunur. Soyut bir sınıfı göstermenin yararlı olabileceğini söyleyen hoca, bunun için boş bir arayüz kutusu koyup onu kopyalayarak soyut olması gereken üst sınıfı gösterdiğini anlattı. Örnek olarak bir `Subject` sınıfına bir gözleyici listesi ve altına `ConcreteSubjectA`, `ConcreteSubjectB` sınıfları ekledi. Çizim `File > Export to image file` ile PNG olarak kaydedilebilir. Hoca kullandığı sürümün en son sürüm olmadığını, aracın basit bir jar dosyası olduğunu ve yeni sürümün eski sürümle çizilmiş şemaları açamadığı için eski sürümde kaldığını söyledi; istenen herhangi bir şema aracı da kullanılabilir.

Örnek Sorular

Soru 1: Termometre ölçümü nasıl bir komut olur? (derste sözlü, 00:20:44)

Akıllı ev kumandası örneğine bir termometre eklenip ondan ölçüm alınmak istenirse, bu ayrı bir komut olarak mı gerçeklenmeli? Şimdiye kadarki komutlardan farkı nedir?

Derste verilen cevap: Ölçüm alma, "açabildiyse `true`, açamadıysa `false`" döndüren komutlardan farklıdır; bir sonuç (derece) döndürmesi gerekir. Kalıp yapısında bu, Receiver'ın bir eyleminin (action) olmasına karşılık gelir: ölçüm sonucunun geri döndürülmesi Receiver nesnesinin kullanılmasıyla sağlanır. Hoca örneğin bu yönde geliştirilebileceğini not etti.

Soru 2: Iterator ile Adapter'ın farkı nedir? (derste sözlü, 01:02:35)

Menü örneğinde her yapı için bir iterator sınıfı yazmak Adapter'a benziyor. Aynı işi Adapter'la yapıp her yapıdan bir `ArrayList` döndüremez miydik?

Derste verilen cevap: Adapter, belli bir metot çağrısı için iki farklı arayüz arasına köprü koyar. Iterator ise kompozit bir varlığın bileşenleri arasında eş biçimli dolaşmayı sağlar; yalnız eş biçimlilik değil dolaşım da sağlar. Problemin odağı yalnız farklı gerçeklemeleri ortak yapı altında toplamak olsaydı Adapter yeterli olurdu; ama farklı kompozit varlıkların eş biçimde dolaşılması gereken bir problemde Iterator kalıbı anılmalıdır. Java kullanılacağı önceden kararlaştırılmışsa "adapter kullanırız, dolaşımı Java'nın iterator'ı halleder" diye bu en azından vurgulanmalıdır (ayrıntı Bölüm 7.5'te).

Soru 3: Caretaker sahte bir memento yükleyebilir mi? (derste sözlü, 01:41:59)

İç sınıf çözümünde (Bölüm 10.1) Caretaker, Originator'a hiç almadığı "sahte durum" gibi bir durumu yükleyebilir mi? Yükleyebiliyorsa nasıl önlenir?

Derste verilen cevap: `new Memento()` ile doğrudan memento üretemez, Memento'nun private `setState` metodunu da çağıramaz. Ama ikinci bir `Originator` oluşturup `modify("sahte durum")` deyip ondan `createMemento()` ile meşru bir memento alabilir ve `subject.setMemento(memFake)` ile asıl nesneye yükleyebilir; derste denendi ve çalıştı. Önlem olarak Originator'a UUID verip mementoya koymak ve `setMemento`'da kimlik eşleşmesini denetlemek, ya da memento listesinin bakımını Originator'a verip Caretaker'ın yalnız `activateMemento(int)` gibi bir numarayla geri dönüş istemesi önerildi (ayrıntı Bölüm 10.2'de).

> **Çözüm doğrulaması:** Senaryo küçük bir Java programıyla denendi: `subject.setMemento(memFake)` sonrasında `System.out.println(subject)` "sahte durum" yazdırır.

Soru 4: "Create and set memento" neden olmaz? (derste sözlü, 01:46:26)

Mementoyu oluştururken aynı anda setleyen tek bir metot yazsak, `setMemento`'yu dışarı açmamıza gerek kalmaz mı?

Derste verilen cevap: Hayır; Originator'ın daha önce alınmış bir mementoya, yani o anki durumuna geri dönebilmesi için mementoyu dışarıdan alması gerekir. Bu yüzden ayrı bir set metodu şarttır. Dışarıdan memento almadan çözüm isteniyorsa mementoların bakımı Originator'a verilir ve Caretaker yalnız numarayla geri dönüş ister.

Soru 5: Müşteri ilişkileri programı için tasarım (derste sözlü, 02:35:38)

Hoca, kendi sorduğu tasarım sorularına benzer bir örnek olarak şu problemi verdi: Bir müşteri ilişkileri programı yazıyoruz. Yeni müşteri eklendiğinde hoş geldiniz e-postası gönderilecek ve ikamet adresinin doğru olup olmadığına bakılacak. Fakat benzer ihtiyaçların tekrar ortaya çıkabileceğini düşünüyoruz; örneğin her yeni müşteri eklendiğinde farklı türde kampanyaların yürütülmesi söz konusu olabilir.

- (a) Bu problemi çözmek için hangi tasarım kalıbını (ya da kalıpları) kullanırsınız?
- (b) Bu kalıbı neden kullanırsınız?
- (c) Kalıbı kullanarak yapacağınız tasarımın UML sınıf şemasını çizin.

Derste verilen cevap (Bölüm 14.2): Observer kalıbı. Her yeni müşteri eklenmesi, önceden sayısı bilinmeyen ve sonradan artabilecek farklı işleri tetiklemelidir; bu işler `Customer`'a dokunmadan yeni gözleyici sınıflarıyla eklenebilir ve gözlenen ile gözleyiciler arasındaki bağlaşım tek bir soyut ilişkiyle sınırlı kalır. Sınıf şeması: `Customer` (`- myObs: Vector<ICObserver> {static}`, `+ attach(ICObserver) {static}`, `+ detach(ICObserver) {static}`, `+ getState()`, `+ notifyObs()`), `«interface» ICObserver` (`+ update(myCust: Customer)`) ve onu gerçekleyen `AddrVerification`, `WelcomeLetter`, `BrickAndMortar` (kampanya) sınıfları; `Customer`'dan `ICObserver`'a yönlü ilişki.

Hocanın bu tür sorulardaki beklentileri:

- Doğru kalıbı seçmek. Bazı sorularda birden fazla kalıbın birlikte kullanılması gerekebilir; en az bir kalıp yerinde önerilmişse ikincisinin anılmaması büyük kayıp değildir. Hocanın düşünmediği bir kalıp önerilirse, gerekçe doğru ve ikna edici anlatılırsa o da doğru kabul edilir.
- Kalıbı neden seçtiğini mantıklı biçimde anlatmak.
- UML sınıf şemasında kalıbın gereksinimlerinin görünmesi: ör. Observer seçildiyse müşteri ile kampanyalar arasında olması gereken gerçekleme/kalıtım ilişkisi ve gözlenenden gözleyiciye ok (tasarım gerektiriyorsa ters yöndeki ok da) şemada olmalıdır.
- Genel tasarım kurallarına uymak: üye alanlar private, üye metotlar public gösterilmeli; private olması gereken kritik bir metot public düşünülmemelidir.
- Kod yazmak zorunlu değildir ve çalışıp çalışmadığına bakılmaz; ama önemli bir ayrıntıyı (ör. somut gözleyicinin `update` metodunun nasıl kodlanacağını) göstermek için birkaç satır kod, ya da anlatımı destekleyecek bir sıralama (sequence) veya aktivite diyagramı eklenebilir.

Dersin Toparlaması

Bu derste davranışsal kalıplardan Command, Iterator, Memento ve Observer işlendi. Command, metot çağrılarını nesneye çevirerek kuyruklama, günlük tutma ya da aynı işi menüye ve düğmeye bağlama gibi ihtiyaçları karşılar; akıllı ev kumandası örneğinde her düğme bir komut nesnesiydi ve Receiver'ın komuta nasıl verileceği tartışıldı. Iterator, bir bileşiğin parçalarını iç yapıyı açmadan eş biçimli dolaşmayı sağlar; iki yemek firmasının dizi ve listeyle gerçeklenmiş menüleri, ortak `Menu` ve `Iterator` arayüzleriyle garsona tek bir kodla sunuldu ve kalıbın Adapter'dan farkı soru-cevapla netleştirildi.

Memento, nesnenin durumunu sarmalamayı bozmadan saklayıp geri yüklemeyi amaçlar; gizliliğin dile göre `friend`, paket ve `sealed` jar ya da iç sınıfla nasıl sağlanabileceği gösterildi ve derste canlı olarak iç sınıf çözümünde başka bir Originator'dan alınan memento ile sahte durum yüklenebildiği, bunun için kimlik denetimi ya da bakımın Originator'a verilmesi gibi önlemler tartışıldı. Observer, gözlenen nesnedeki değişikliği bağlaşımı artırmadan sayısı bilinmeyen gözleyicilere bildirir; pull ve push modelleri, bildirimi kimin tetikleyeceği ve müşteri ilişkileri örneği üzerinden kalıbın çok sayıda gerçekleme kararı gerektirdiği görüldü. Hoca, kalan davranışsal kalıpların (Strategy, State, Template Method, Visitor, Mediator) sonraki derslerde işleneceğini söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.